High-Level Design
Services, Stack and How They Connect

What IFMS is made of, why each piece was chosen, and how the seven modules, the database and the two portals fit together. Written to be approved before the first line of production code; the Low-Level Design follows it.

Integrated Financial Management System · ABVMU, Lucknow · continues IFMS-DG-ACC-005 v1.0

Document ID	IFMS-DG-HLD-006
Version	0.1 · 14 October 2026 · draft for approval
Prepared by	Vaibhav, Software Developer
Reviewed by	_____ (name, role, date)
Approved by	_____ (name, role, date)
Source documents	The seven BRDs treated as approved from 12 Oct 2026 (Budget · HRMS & Payroll · Finance & Accounting · Cash & Bank · Affiliation & Examination Fee Revenue Import · Examination Management & Payment · Inventory, Stores & Asset Depreciation) · IFMS-DG-ACC-002 v1.0 · IFMS-DG-API-004 · IFMS-DG-ACC-005 v1.0 · ABVMU Scope of Work
Classification	Internal · developer, reviewer and BA reference

0 How to use this document

ACC-005 explained what the system must do with money. This document explains what the system is **made of**. It is written to be approved: every choice that costs money, locks in a technology or would be expensive to reverse later is a numbered decision in Part D with the alternatives that were considered, and the few things only the reviewer can settle are listed in Part F. Nothing in this document changes a BRD rule; where a rule shapes the design, its ID is quoted.

1. **Draw the system first.** Part A has four pictures: who talks to IFMS, what it is built from, how the seven modules sit around Cash & Bank and Finance, and one payment followed from a click to the ledger.
2. **Name every technology and say why.** Part B is the stack, one layer at a time, with the alternatives that were rejected and the reason.
3. **Show how the pieces are linked.** Part C covers deployment, integrations, module-to-module calls, the shared kernel, data, security, non-functional targets and the delivery pipeline.
4. **Record the decisions and the order of work.** Part D is the decisions register (16 entries); Part E is the build order; Part F is what needs the reviewer.

0.1 Contents

Part A — The system in one picture

A1 Who talks to IFMS · A2 What it is built from · A3 Inside the API: modules and the shared kernel · A4 One payment, from click to books

Part B — The stack and why

B1 Summary · B2 Backend · B3 Database · B4 Persistence and migrations · B5 Frontend · B6 Security · B7 Supporting services · B8 Build, test and tooling

Part C — How the pieces are linked

C1 Deployment · C2 Integration points · C3 Module-to-module communication · C4 Shared kernel · C5 Data · C6 Security · C7 Non-functional targets · C8 Delivery pipeline

Part D — Decisions register

D-01 to D-16, each with reasoning, alternatives, consequences and status

Part E — Build order

Seven phases, what each delivers, what must be answered first

Part F — Needs sign-off

Ten points only the reviewer can settle, each with a proposed default

Appendix

Technical glossary · document family

Box	Meaning
`IN SIMPLE WORDS`	The idea without jargon.
`DECISION`	A numbered choice (D-01 …). Status `Proposed` until the reviewer marks it `Approved`.
`WHY`	The reasoning behind a choice, specific to this system.
`ALTERNATIVES CONSIDERED`	What else was looked at and why it was not chosen.
`DEVELOPER NOTE`	How it will be built. Detail belongs to the LLD; only what shapes the design is here.
`WATCH OUT`	A trap that would cause a money bug, an audit objection or rework.
`NEEDS SIGN-OFF`	The reviewer must decide; listed with its ID (F-01 …) in Part F.

> **WHAT THIS DOCUMENT DOES NOT DO**
>
> This document contains **no table definitions, no endpoint list and no screen designs**. Those belong to the Low-Level Design (IFMS-DG-LLD-007), which is written only after this document is approved, so that the LLD is not built on a stack that is then changed. Library versions quoted are the current releases at the time of writing (October 2026); the LLD pins them.

0.2 How this relates to ACC-005 and API-004

Three earlier documents touch the design. Nothing here contradicts them; where the thinking has moved, the row says so.

Topic	Earlier document said	This document	See
Service boundaries	API-004 names four services: finance-api, hr-api, academic-api, ops-api.	The same four boundaries, kept as **module groups inside one deployable**. Posting inside Finance stays an in-process call, as API-004 already required.	A3, D-01, F-02
Events between services	API-004: events from hr-api, academic-api and ops-api are consumed idempotently by finance-api.	Same contract, delivered through an in-database event registry instead of a broker. Consumers remain idempotent.	C3, D-04
Posting rules	API-004 GL §2: posting rules are versioned data, approved by Finance.	Carried forward unchanged.	C4, D-16
General Ledger	ACC-002 v1.0: no GL in scope. ACC-005: Finance & Accounting BRD makes the GL the destination of everything.	The GL is a core module and the hub of Part A.	A3, C5
Three records of one rupee	ACC-005 A3: Budget, Bank and Books are three separate append-only stores, linked by source document.	Becomes three ledgers in three schemas with a nightly tie-out (D-08).	C5
Queues and caches	Earlier stack notes assumed Redis and a job queue.	Not in phase 1; PostgreSQL carries jobs, locks and the outbox.	B7, D-12

0.3 At a glance

1
deployable backend, seven modules inside

2
portals (ERP and Examiner), one web app

3
append-only ledgers: budget, books, stock

16
decisions for approval, 10 points for sign-off

READING ORDER

Fifteen-minute read for a reviewer: the four figures in Part A, the summary table B1, the decisions in Part D and the sign-off list in Part F. The rest is reference for the LLD.

Part A — The system in one picture

Four pictures, from the outside in: who uses IFMS, what it is built from, how the modules are arranged, and one payment followed end to end.

A1 Who talks to IFMS

> **IN SIMPLE WORDS**
>
> IFMS is one web application used from the university's browsers. Internal staff log in to the ERP portal; external examiners get a smaller portal of their own, reached from the duty order they receive by e-mail. The system **receives** fee payments from the existing university portal, **produces** payment files and challans for the banks and statutory bodies, **sends** e-mail, and once, at cut-over, **takes** the opening balances from Tally. Nothing else connects to it.

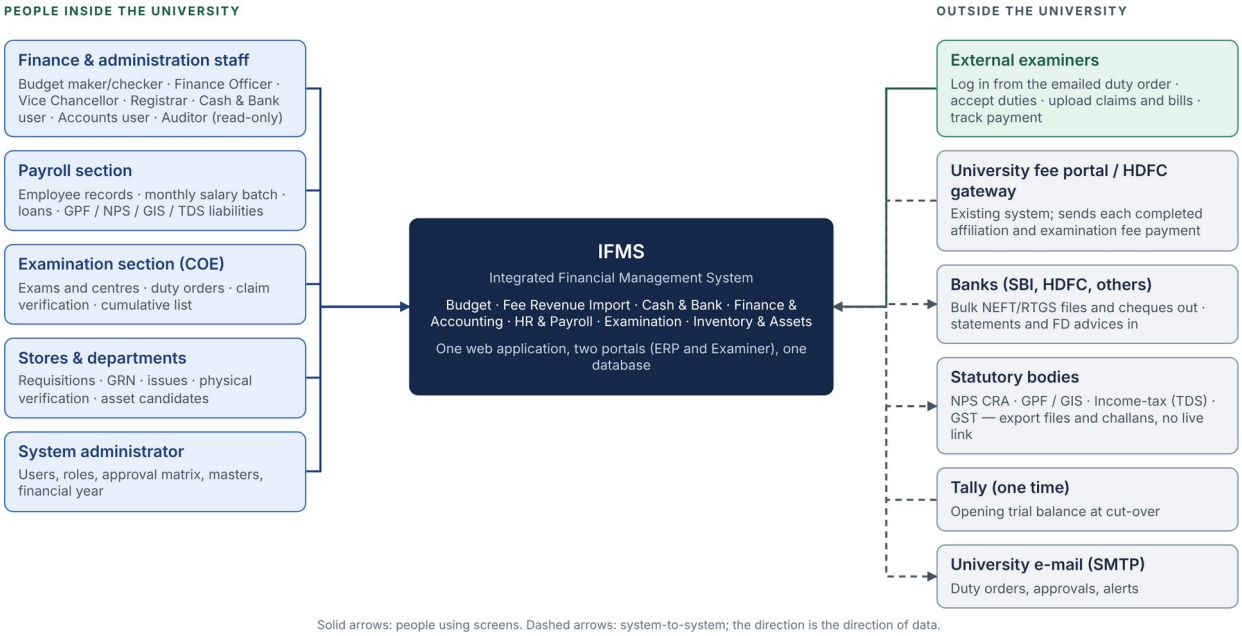


Figure A1 — System context. Everyone and everything that touches IFMS. There is no live link to a bank or a statutory portal in phase 1; those exchanges are files prepared by IFMS and uploaded by the user.

Actor	What they do in IFMS	Portal	From BRD
Budget maker / checker	Fund receipts, allocation per head, quarterly revision, re-appropriation, year-end closing proposal	ERP	`BUD`
Finance Officer (FO)	Verifies salary batches, bills, claim lists and closings; forwards to the VC and on to Cash & Bank	ERP	`HR §15` `BUD §19`
Vice Chancellor (VC)	Final approval of salary batches, allocations, re-appropriation, deviations above a head	ERP	`HR-FR-014` `BUD-BR-008`
Registrar	Approval matrix owner; overrides on examiner capping where the policy allows	ERP	`EXM §13` `Q-X-03`
Cash & Bank user	Payment vouchers, cheque / UTR capture, bank accounts, FDs, interest, challans, statement upload	ERP	`CB`
Payroll user	Employees, service book, monthly salary batch, loans, liability summaries	ERP	`HR`
Accounts user / Finance verifier	Chart of accounts, manual journal vouchers, opening balances, period close, statements	ERP	`FIN`

Actor	What they do in IFMS	Portal	From BRD
Examination section (COE)	Exams, centres, examiner onboarding, duty orders, claim verification, cumulative list	ERP	`EXM`
Storekeeper / department user	Requisitions, GRN, issues, returns, physical verification, asset candidates	ERP	`INV`
Receivables / Fee user	Import runs, exception queue, manual allocation of unmatched receipts	ERP	`FEE`
Auditor	Read-only across every module, with drill-down from any statement figure to the source document	ERP	`FIN-FR-015`
System administrator	Users, roles, approval matrix, masters, financial year, configuration	ERP	—
External examiner	Views assigned duties, accepts the duty order, uploads claims and bills, tracks payment	Examiner	`EXM §8.5` `EXM §10.3`

WATCH OUT

- The examiner portal is reachable from the public internet by design. It must therefore share **nothing** with the ERP routes except the login mechanism: its own layout, its own middleware rule, and an API role that can see only the examiner's own records.
- Statutory bodies (NPS CRA, GPF, GIS, Income-tax, GST) have no API here. IFMS produces the file or challan in the required format; a person uploads it. The format samples are an open question (`Q-X-05`).
- The bank link is file-based in phase 1 (`Q-CB-03`). A payment is "paid" in IFMS when the user records the UTR or cheque number, not when a bank API says so.

A2 What the system is built from

IN SIMPLE WORDS

Five boxes. The **browser** runs the screens. The **web app** serves them and keeps the login cookie. The **API** holds every business rule and is the only thing that touches the database. The **database** holds the facts. The **file store** holds the PDFs and scans. Nothing else writes money.

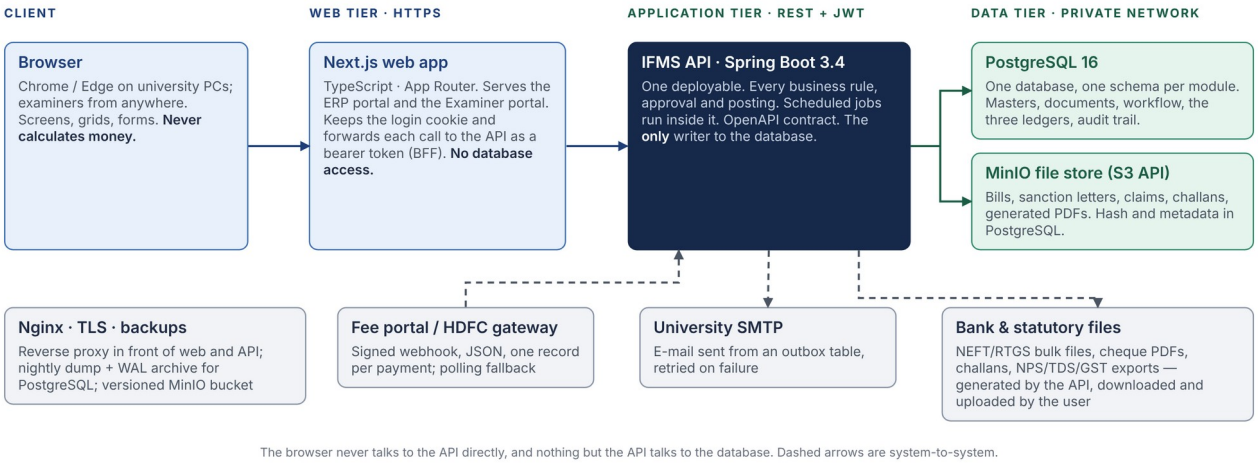


Figure A2 — Containers. One web application, one API, one database, one file store, plus the three external exchanges from Figure A1.

Container	Technology	Responsibility	Talks to
Browser	Chrome / Edge, current versions	Screens, data grids, forms, client-side preview sums only	Web app over HTTPS

Container	Technology	Responsibility	Talks to
Web app	Next.js 15 (App Router), TypeScript	Serves both portals; holds the session cookies; forwards each API call with a bearer token; route protection per portal	API over HTTPS
API	Spring Boot 3.4, Java 21, one deployable	All business rules, workflow, posting, reports, scheduled jobs, OpenAPI contract	PostgreSQL, MinIO, SMTP, fee portal (inbound)
Database	PostgreSQL 16	Masters, documents, workflow state, the three ledgers, audit trail, outbox, job locks	API only
File store	MinIO (S3 API)	Attachments and generated documents; versioned; retention policy	API only (pre-signed links to the browser)
Reverse proxy	Nginx	TLS, routing, rate limits on login and the webhook	Everything in front

DEVELOPER NOTE

- The API is **one process**. Scheduled jobs (nightly tie-out, interest accrual, outbox delivery, utilisation alerts, FD maturity reminders) run inside it under a database lock so that a second instance never runs the same job twice. A second instance is a configuration change, not a code change: the API is stateless and the token is self-contained.
- The web app never opens a database connection. If a screen needs data, there is an endpoint for it. This keeps one audit trail and one set of rules.
- MinIO is reached only by the API. The browser uploads and downloads through short-lived pre-signed links, so large PDFs never stream through the API process.

A3 Inside the API: modules and the shared kernel

IN SIMPLE WORDS

Each module is a folder with a **front door** (its public Java interface and REST endpoints) and **locked rooms** (its tables and internals). A module may knock on another module's front door; it may never walk into its rooms. The build checks this rule, so the boundary cannot erode quietly. Cash & Bank and Finance are the two hubs: every source module hands its approved document to Cash & Bank for payment, and every money event ends in the General Ledger.

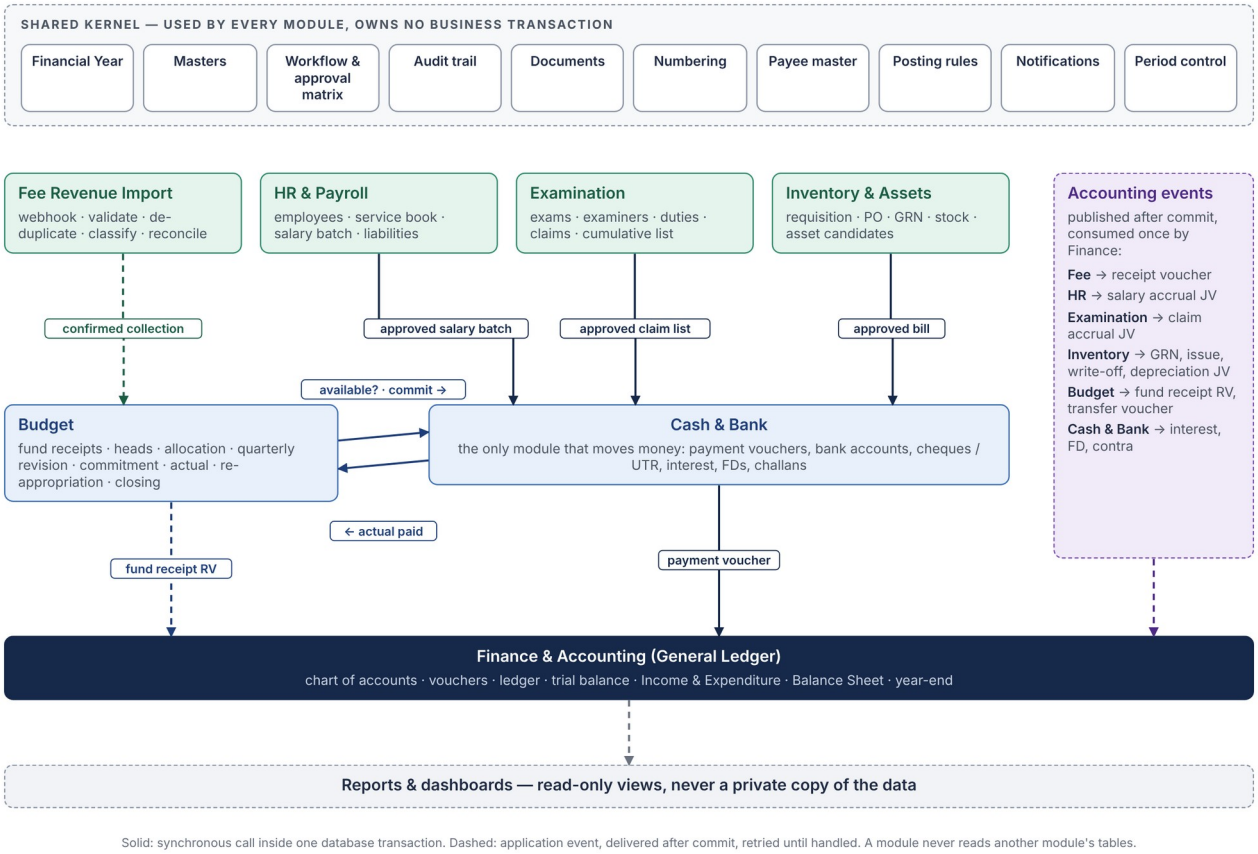


Figure A3 — Module map. The arrows are the twelve hand-offs of ACC-005 Figure A2, now classified: solid ones must be atomic and are synchronous calls; dashed ones are events delivered after commit (see C3).

Module	Java package	API-004 group	Owns (creates the original record)	Calls the front door of
Budget	ifms.budget	finance	Fund receipts, allocation per head, revisions, re-appropriation, commitments, actuals, closing	Finance (events)
Fee Revenue Import	ifms.fee	finance	Imported fee transactions, import runs, exceptions, classification	Budget, Finance (events)
Cash & Bank	ifms.cashbank	finance	Bank/cash accounts, payment vouchers, cheques, UTRs, interest, FDs, challans, statement lines	Budget, Finance (sync)
Finance & Accounting	ifms.finance	finance	Chart of accounts, posting rules, vouchers, ledger lines, periods, statements, opening balances	—
HR & Payroll	ifms.hr	hr	Employees, service book, pay structures, salary batches, deductions, liabilities, loans	Budget (sync), Cash & Bank (sync), Finance (events)
Examination	ifms.exam	academic	Academic masters, exams, centres, examiners, duties, claims, cumulative lists	Budget (sync), Cash & Bank (sync), Finance (events)
Inventory & Assets	ifms.inv	ops	Items, stores, requisitions, POs, GRNs, stock movements, asset candidates, depreciation runs	Cash & Bank (sync), Finance (events)

Module	Java package	API-004 group	Owns (creates the original record)	Calls the front door of
Shared kernel	ifms.core	all	Financial year, masters, workflow, audit, documents, numbering, payee master, posting-rule engine, notifications, period control	Used by every module

WATCH OUT

- The four API-004 names survive as **groups**, not processes. If the project ever has to split the deployable, the finance group (Budget, Fee, Cash & Bank, Finance) must stay together: those four share one transaction on every payment.
- Reports and dashboards read through views and never keep a private copy of any figure (`FIN-FR-015`). A dashboard counter that is stored separately is a counter that drifts.

A4 One payment, from click to books

IN SIMPLE WORDS

The Cash & Bank user confirms a payment and types the UTR. In the time it takes the screen to refresh, the system has checked who they are, checked that nobody did this already, locked the budget head, recorded the actual, posted a balanced voucher, written the audit row and queued the e-mail. All of that is one database transaction: either everything is written or nothing is.

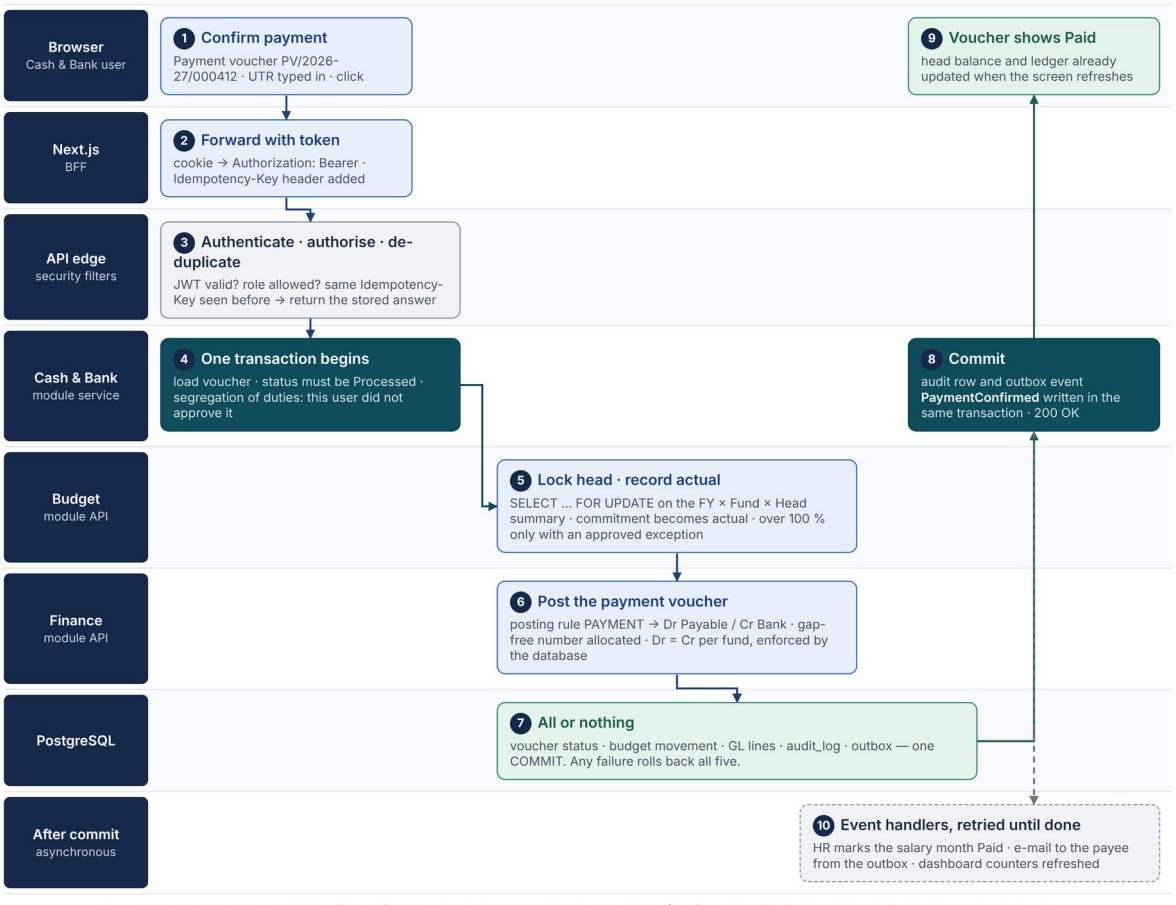


Figure A4 — The life of one payment inside the system. Steps 4 to 8 are one transaction; step 10 happens after it and is retried until it succeeds.

#	What happens	What can go wrong, and what the user sees
3	Authenticate, authorise, de-duplicate	Token expired → web app refreshes it silently and retries once. Wrong role → 403, logged. Same Idempotency-Key → the stored answer is returned; a double click never pays twice.
4	Segregation of duties	The confirming user verified or approved this voucher → 409 “you have already acted on this document” (`GAP-22`).
5	Budget head locked and charged	Another user is charging the same head → waits up to 3 s for the row lock, then 409 “head busy, retry”. Head exhausted → 422 “Non-Salary (Govt): available ₹28,20,000, requested ₹30,00,000” and nothing is written (`BUD-FR-025`).
6	Voucher posted	Posting rule missing or unbalanced → the database rejects the voucher at commit (deferred constraint, `FIN-FR-006`); the whole transaction rolls back and the error names the rule.
7	Commit	Database unreachable → 503; nothing written; the user retries with the same key.
10	After commit	SMTP down → the e-mail stays in the outbox and is retried with back-off; the payment is still paid. HR handler fails → retried; the event is never lost because it was committed with the payment.

WATCH OUT

Nothing in the browser calculates money that is then trusted. The screen may show a preview (“net payable ₹14,750”), but the service recomputes every figure from masters and rates inside the transaction. This is the rule that keeps the ACC-005 Part D year reproducible by a test.

Part B — The stack and why

One layer at a time. Each choice names the alternative that was closest and the reason it lost. The full reasoning sits in the decisions register (Part D); this part is the readable version.

B1 Summary

Layer	Choice	Why, in one line	Decision
Language, runtime	Java 21 (LTS)	Exact decimal arithmetic, strict compilation, long-term support, common skill among government IT vendors	D-02
Framework	Spring Boot 3.4	Declarative transactions and locking, security, validation, scheduling, mature ecosystem	D-02
Module boundaries, events	Spring Modulith	Boundaries verified in the build; in-database event registry doubles as the outbox	D-04
Database	PostgreSQL 16	NUMERIC money, deferrable constraints for Dr = Cr, row locks, JSONB, free	D-03
Persistence (writes)	Spring Data JPA, Hibernate 6	Object graphs with optimistic and pessimistic locking	D-13
Persistence (reports)	SQL through Spring JdbcClient; jOOQ in reserve	Statements are set-based SQL; reviewable by the BA	D-13
Schema migrations	Flyway, forward-only; Hibernate validates	Version-controlled SQL with checksums; the app refuses to start on drift	D-09
Audit	Hibernate Envers + audit_log table	Field-level history with one annotation; business trail in the same transaction	D-10
Security	Spring Security 6, JWT; cookies via the BFF	Stateless API; tokens never readable by browser scripts	D-06
API contract	springdoc-openapi → generated TypeScript client	Java DTOs and frontend types cannot drift; money travels as strings	D-05
Frontend	Next.js 15, TypeScript, Tailwind, shadcn/ui, TanStack Table and Query, React Hook Form, Zod	Two portals in one codebase; grids for register-heavy screens	D-05
Attachments	MinIO (S3 API)	Versioned object store; swap for any S3 if hosting changes	D-11
Documents out	JasperReports (PDF), Apache POI (Excel)	Pixel-exact vouchers, cheques, challans, UCs, registers	D-14
E-mail	Spring Mail over the university SMTP, outbox table	Never lost, never sent twice	D-12
Jobs	Spring scheduling with ShedLock	Dozen small jobs in the API process; safe with two instances	D-15
Cache, queue	None in phase 1	PostgreSQL carries locks, idempotency keys and the outbox at this volume	D-12
Build, CI	Maven wrapper, GitHub Actions, Docker	One command builds and tests; the image that passed UAT is promoted	C8
Tests	JUnit 5, Testcontainers (real PostgreSQL), Modulith verification, MockMvc; the ACC-005 Part D year as a fixture	Tests run against the real database engine and the real rules	B8

Layer	Choice	Why, in one line	Decision
Developer tooling	Claude Code with the ECC plugin; CLAUDE.md house rules; Java and TypeScript rule packs	AI-assisted code held to the same rules as hand-written code	B8

B2 Backend: Java 21 and Spring Boot 3.4

IN SIMPLE WORDS

The backend is the part of the system that is allowed to be slow to write and must be impossible to get wrong. Java makes exact money arithmetic, transactions and audit history ordinary features of the language and the framework rather than things the team builds and polices by hand.

WHY

- **Money is exact.** BigDecimal with an explicit rounding mode is the only numeric type the code is allowed to use for amounts; a floating-point number cannot compile into an amount field. ACC-002 C7 rounding order becomes a one-line rule.
- **Transactions are declared, not coded.** One annotation marks a service method as atomic; isolation and a pessimistic row lock map straight onto PostgreSQL's SELECT … FOR UPDATE. The payment in Figure A4 is one method.
- **History is one annotation.** Hibernate Envers records every change to an audited entity with user and time; the BRDs' old value / new value requirement is met without hand-written logging.
- **It is the common stack of the client's world.** Government and PSU support vendors in Uttar Pradesh and nationally staff Java teams routinely; hand-over and long-term maintenance (F-06) are easier than with a newer stack.
- **The compiler is a reviewer.** With AI-assisted development (B8), a strict compile and test gate catches a wrong type, a missing null check or a broken contract before anything reaches UAT.

ALTERNATIVES CONSIDERED

NestJS (TypeScript). Fastest start for a team that already writes React; one language end to end. Rejected because JavaScript's number type makes money a discipline rather than a guarantee, audit history must be hand-built, and the stack is uncommon among the client's likely support vendors.

Go. Excellent for high-throughput services. Rejected because this system is roughly eighty relational tables of CRUD, workflow and reporting, which Go makes verbose, and it lacks a mature ORM and audit library.

Python (Django / FastAPI). Decimal is exact and the admin is a gift. Rejected on typing and concurrency for payroll runs and report generation, and on vendor familiarity.

.NET. Technically equivalent. Not chosen because Linux-first hosting and the team's skills favour Java; nothing in this design would be impossible in .NET.

WATCH OUT

Java has well-known traps that cost money in a finance system: lazy loading inside a loop (N+1), open-session-in-view leaking transactions into the web layer, and entity equality bugs. These are not left to memory: they are rules in CLAUDE.md and the ECC Java rule pack, and the build fails on the ones that can be detected.

B3 Database: PostgreSQL 16

IN SIMPLE WORDS

Every fact the auditor will ever ask about lives in one PostgreSQL database. It is chosen because it can enforce accounting rules itself, so that a bug in the application cannot write an unbalanced voucher.

WHY

- **NUMERIC(18,2)** stores amounts as exact decimals; it maps one-to-one onto BigDecimal.
- **Deferrable constraint triggers** check that debits equal credits per voucher and per fund at commit time, after all lines are written (`FIN-FR-006`). This is the single most valuable feature for this system and the one most other databases lack.
- **Row-level locks** on the budget summary row stop two approvers from overdrawing the same head, while readers of the dashboard are never blocked.
- **JSONB** holds the raw fee payload from the portal and the old/new snapshots in the audit trail, indexed where it matters, without turning the schema into a document store.
- **Recursive queries** roll the chart of accounts (Account Class → Ledger Group → Ledger Head → Sub-ledger, `FIN §5.2`) into statements in one query.
- **Partial unique indexes** express business rules the BRDs state in words: one open financial year, one active pay structure per employee, one import record per source transaction (`FEE-API-009`).
- **Backups are boring.** Nightly dump, continuous WAL archive, point-in-time recovery; free; runs on any VM the university or the state data centre provides.

ALTERNATIVES CONSIDERED

MySQL / MariaDB. Familiar and free. Rejected for the lack of deferrable constraint triggers and weaker JSON indexing.

Oracle / SQL Server. Strong transactional engines used by banks. Rejected on licence cost and container-unfriendliness; revisited only if the hosting location already provides one (F-01).

MongoDB. Rejected: no referential integrity, multi-document transactions are slow, and a trial balance over documents is a reporting nightmare.

B4 Persistence, migrations and audit

The team touches the database in two distinct ways, and each has one tool.

Need	Tool	Rule
Commands: create, verify, approve, post, pay	Spring Data JPA (Hibernate 6)	Entities per module; version column on every workflow document; pessimistic lock on summary rows; lazy loading by default; no open-session-in-view
Reports: trial balance, I&E, Balance Sheet, registers, Budget vs Actual	SQL files run through Spring JdbcClient	Reads only; one named SQL per report; reviewable by the BA and the accountant; jOOQ adopted only if the SQL grows past what plain files can manage (D-13)
Schema changes	Flyway, versioned SQL per schema	Forward-only; immutable after it has run on UAT; expand/contract for renames; Hibernate set to validate, never to update (D-09)
History of masters and documents	Hibernate Envers	@Audited on every master and workflow entity; revision carries user, role and time (D-10)
Business audit trail	audit_log table in core	Written by the workflow engine in the same transaction: who, role, action, document reference, old → new (JSONB), remarks, IP, time
Ledger integrity	Database constraints and triggers	Posted rows cannot be updated or deleted (trigger); Dr = Cr per fund deferred to commit; numbering gap-free per series (`FIN §25`)

WATCH OUT

- Hibernate's automatic schema update is **forbidden** in every environment, including developer laptops, so

that nobody learns to rely on it.

- The three ledgers (budget movements, GL lines, stock movements) are append-only at the database level, not just by convention: an UPDATE or DELETE on a posted row fails in the database regardless of which code path tried it (`BUD-BR-008` `FIN-BR-012` `CB-BR-008` `INV-BR-007`).

B5 Frontend: Next.js for both portals

IN SIMPLE WORDS

The screens are a separate TypeScript application built with React through Next.js. It serves two portals from one codebase, keeps the login cookie out of reach of browser scripts, and gets its data types generated from the API so that a renamed field in Java fails the frontend build instead of a UAT session.

WHY

- **Two portals, one codebase.** The ERP and the Examiner portal are two route groups with separate layouts and separate middleware rules. One deployment, one design system, two front doors.
- **A server layer for security.** The web app holds the session cookies and forwards calls to the API with a bearer token (D-06). The browser never sees a token.
- **Types from the contract.** springdoc generates the OpenAPI document from the Spring controllers; the frontend build generates the TypeScript client from it. Money fields are strings on both sides; preview sums use a decimal library, never the JavaScript number type.
- **Register-heavy screens.** About seventy per cent of the seven BRDs are filterable registers (salary register, cumulative remuneration list, trial balance, stock register, exception queue). TanStack Table handles sorting, filtering, grouping and export; shadcn/ui and Tailwind give an accessible, unlicensed design system; React Hook Form with Zod handles the multi-row voucher and claim forms.
- **Fast first paint on slow links.** Server rendering matters for staff on the university network and examiners on mobile data.

ALTERNATIVES CONSIDERED

React with Vite (single-page app). Simpler hosting, since the output is static files. Rejected because the token would have to live in the browser and there would be no server layer for cookies and route protection.

Server pages inside Spring (Thymeleaf). One deployable, no Node. Rejected because the data grids these BRDs need (keyboard navigation, inline filters, thousands of rows) are a poor fit for server-rendered pages.

Angular. Equally capable. The team's React experience decided it; nothing in the design depends on React specifically.

WATCH OUT

- The web app contains **no business rule and no database access**. Every validation that matters is repeated in the API; the frontend copy is for the user's convenience only.
- Indian number formatting (₹1,12,50,000.00), the April–March financial year and Asia/Kolkata time are in one shared formatting module, used everywhere, tested once.

B6 Security stack

The components; how they fit together is drawn in C6.

Concern	Choice
Authentication	Username and password with bcrypt; JWT access token (15 min) and rotating refresh token (8 h) issued by the API; stored by the web app as httpOnly, Secure, SameSite=Strict cookies. University SSO can be added later behind the same interface.

Concern	Choice
Authorisation	Roles from the BRDs (A1 table) enforced by Spring Security method rules on every endpoint; deny by default. Row-level rules (an examiner sees only their records) enforced in the service, not the controller.
Segregation of duties	The workflow engine, not the role model: one person may act at one step of one document, whatever roles they hold (`GAP-22`).
Sensitive data at rest	Bank account numbers encrypted with AES-GCM, key held outside the database; PAN masked in lists; Aadhaar not stored at all.
Transport	TLS 1.2+ at Nginx; API, database and file store on a private network; nothing but 443 exposed.
Input and files	Bean Validation on every request body; upload type, size and virus-scan hook; HTML never rendered from user input.
Abuse control	Rate limits on login and the fee webhook; account lock after repeated failures; webhook signature (HMAC) verified before the payload is parsed.
Secrets and dependencies	Secrets from the environment, never in git; dependency vulnerability scan in CI; images rebuilt monthly for base-image patches.
Checklist	OWASP Application Security Verification Standard level 2 used as the review list before UAT.

B7 Supporting services

Service	What it does here	Why this one	Phase
MinIO	Attachments and generated documents; versioning; 10-year retention (`Q-X-04`)	S3-compatible, so the same code works against any provider's object storage if hosting moves; pre-signed links keep files out of the API process	Phase 1
University SMTP	Duty orders, approval requests, alerts, examiner credentials	Already exists; an outbox table in PostgreSQL makes delivery reliable without a queue service	Phase 1
JasperReports, Apache POI	PDF vouchers, cheques, challans, duty orders, salary slips, Utilisation Certificates; Excel registers and statutory exports	Mature, free, same JVM; layouts are files the BA can review	Phase 1
ShedLock	Database lock around each scheduled job	Lets two API instances run safely without a separate scheduler	Phase 1
Spring Boot Actuator	Health, metrics, readiness for the reverse proxy and backups	Built in	Phase 1
Prometheus + Grafana (or the host's own monitoring)	Dashboards and alerts on latency, errors, job failures, disk	Standard; decided with the hosting location (F-08)	Phase 2
Redis	Shared cache, faster rate limits	Not needed at this volume; the triggers that would bring it in are listed in D-12	Deferred
Message broker (Kafka, RabbitMQ)	Cross-service events	There is one process; the in-database event registry does this job	Deferred
e-Sign / DSC service	Signed duty orders, cheques, vouchers (`Q-X-02`)	Printed and signed in phase 1; provider chosen with the client	Phase 2

B8 Build, test and tooling

> **IN SIMPLE WORDS**

Every change goes through the same gate: compile, unit tests, integration tests against a real PostgreSQL started by the test itself, boundary checks, and a replay of the ACC-005 Part D year. A change that breaks a rupee in that year cannot be merged.

Level	Tool	What it proves
Unit	JUnit 5, AssertJ, Mockito	Calculations: pro-ration, rounding order, capping bands, TDS thresholds, depreciation
Data access	Testcontainers with PostgreSQL 16	Queries, locks, constraints and Flyway migrations against the real engine, not an in-memory substitute
Module	Spring Modulith application-module tests	One module boots alone with its dependencies stubbed; events are published and consumed as declared
Boundaries	Modulith verification (ArchUnit underneath)	No package reaches into another module's internals; the build fails otherwise
API	MockMvc, OpenAPI snapshot diff	Endpoints, validation messages, role rules; an unintended contract change is flagged
Year replay	Fixture built from ACC-005 Part D	Thirty-two vouchers in, trial balance, I&E, Balance Sheet and budget view out, equal to the rupee
Frontend	TypeScript, ESLint, Playwright for the critical flows	Generated client compiles; the salary-batch and payment flows work end to end

DEVELOPER NOTE

Development uses Claude Code with the ECC plugin installed in the repository (project scope), with the common, Java and TypeScript rule packs. A CLAUDE.md at the repository root holds the house rules that the AI assistant and every reviewer apply:

- Money is BigDecimal in Java, NUMERIC(18,2) in PostgreSQL and a string in JSON; rounding mode is always explicit.
- Budget is allocated and checked only at Budget Head level; an Expense Head is a classification tag on a bill (`BUD-BR-001` `BUD-BR-002`).
- The three ledgers are append-only; a correction is a linked reversal.
- Any method that charges a head, posts a voucher or moves stock takes the summary-row lock first.
- One person, one step, one document: the workflow engine checks the actor list, not just the role.
- A change is complete only when compile, tests and the year replay pass locally.

Part C — How the pieces are linked

Where things run, what crosses the boundary of the system, how modules talk, what the shared kernel gives everyone, how data is laid out, how the system is secured, what it must achieve, and how a change reaches production.

C1 Deployment

IN SIMPLE WORDS

Two virtual machines: one runs the web app, the API and the reverse proxy; the other runs the database and the file store. Everything is a container started from one compose file. A developer runs the same containers on a laptop; UAT runs them on one machine; production runs them on two.

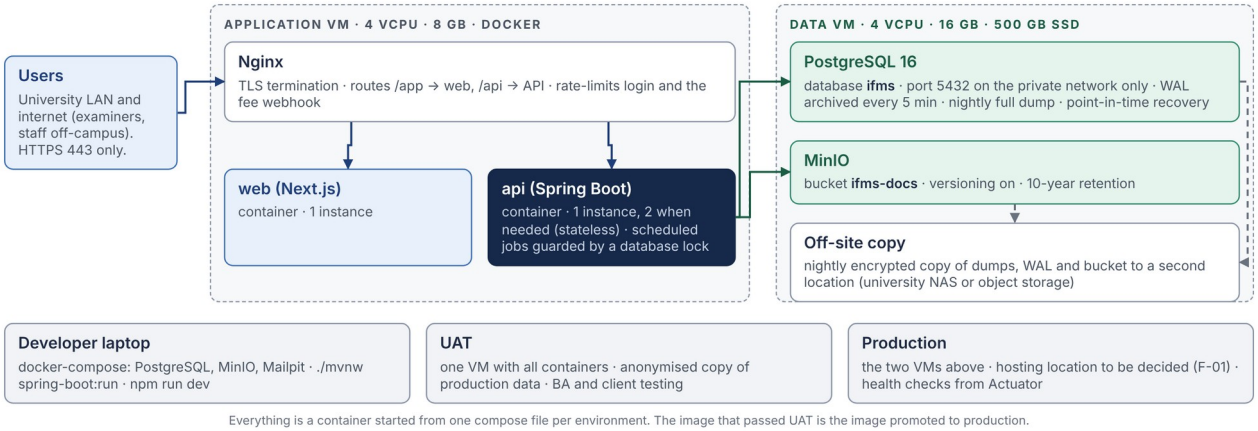


Figure C1 — Deployment. Sizes are proposed starting points for a university of this size (hundreds of employees, thousands of fee records a month, tens of concurrent users); they are confirmed with the hosting decision (F-01).

Environment	Where	Data	Purpose
Developer	Laptop: docker-compose with PostgreSQL, MinIO and a mail catcher	Seed data plus the Part D year	Daily development; the year replay runs here before every push
UAT	One VM, all containers	Anonymised copy of production once it exists; migrated Tally balances before go-live	BA and client testing; every phase demo
Production	Two VMs as in Figure C1; location per F-01	Live; nightly dump, 5-minute WAL archive, off-site copy	Go-live after phase 6

DEVELOPER NOTE

- Recovery objectives proposed: **RPO 15 minutes** (WAL archive) and **RTO 4 hours** (restore the two VMs from images and the latest dump plus WAL). Both are rehearsed once before go-live and once a year.
- The same container image that passed UAT is tagged and promoted; nothing is rebuilt for production.
- A second API container is the first scaling step and needs no code change (stateless, job locks in the database). A read replica for reports is the second; neither is expected in the first two years.

C2 Integration points

Everything that crosses the boundary of the system, with how it fails and what the BRDs still owe.

System	Direction	How	Failure handling	Open point
Fee portal / HDFC gateway	In	Signed webhook (HMAC) posting one JSON record per completed payment to a dedicated endpoint; polling fallback for missed calls; raw payload kept	Duplicate source id → ignored, counted (`FEE-API-009`); malformed → exception queue, never dropped; endpoint rate-limited	`Q-FEE-01` spec and auth
Bank statement (Excel / CSV)	In	User uploads the statement; IFMS matches UTRs, suggests fee heads, blocks reused UTRs	Unmatched lines stay in the queue with the file reference	`Q-FEE-03` `Q-CB-11`
Banks: payments	Out	Bulk NEFT/RTGS file in each bank's format; cheque PDF for printing; HDFC API in phase 2	File generated from Paid-pending vouchers; the UTR is keyed back; a returned payment reverses the voucher and the actual (`Q-CB-05`)	`Q-CB-03` formats
Statutory exports	Out	NPS CRA file, GPF/GIS challans (PDF), TDS and GST return data (Excel)	Regenerated on demand from the liability register; each run logged	`Q-FIN-10` `Q-X-05`
E-mail (SMTP)	Out	Outbox table; templates per event; attachments by link	Retried with back-off; failures visible on an admin screen	—
Tally	In, once	Trial balance export imported as the opening balance voucher (`FIN §19`); party-wise detail for control accounts	Must balance before it can be approved; rows mapped to approved ledgers only	`Q-FIN-07` `Q-X-01`
e-Sign / DSC	Out	Phase 2 provider; documents printed and signed until then	—	`Q-X-02`

WATCH OUT

Every inbound record keeps its raw form (JSONB) next to the fields IFMS extracted from it. When a BA asks why a receipt was classified as Examination income, the answer is in the row, not in a log file that has rotated.

C3 How modules talk to each other

IN SIMPLE WORDS

Two ways, and the choice is never left to taste. If the receiver's work must succeed together with the sender's (a payment charging the budget), the sender **calls** the receiver and both share one transaction. If the receiver can act a moment later (Finance booking an accrual, HR marking a month paid), the sender **publishes an event** that is stored with its own transaction and delivered after commit, as many times as needed until the receiver confirms it.

#	Hand-off (ACC-005 A2)	Mechanism	What travels, and why this mechanism
1	Fee Import → Budget	Event	Confirmed net collection per import run. Budget position may lag the import by seconds; no payment depends on it in the same instant.
2	Fee Import → Finance	Event	One receipt voucher per confirmed receipt. Idempotent on source transaction id.
3	HR → Budget	Sync	Availability check at the FO step and commitment at VC approval. The approval must fail if the head cannot be reserved (`Q-BUD-03`).

4	HR → Cash & Bank	Sync	Approved salary batch becomes a payment instruction in the same transaction as the FO forward, so a batch is never “approved but unknown to Cash & Bank”.
5	Examination → Budget	Sync	Availability and commitment when the cumulative list is approved.
6	Examination → Cash & Bank	Sync	Locked claim list, net of TDS, as a payment instruction.
7	Inventory → Finance	Event	GRN, issue, return, write-off, capitalisation and depreciation journals. Stock is correct immediately; the books follow within the same second in practice.
8	Budget ⇄ Cash & Bank	Sync	Head availability before payment; Cash & Bank selects account, fund, head and expense head (`BUD-FR-023`).
9	Cash & Bank → Budget	Sync	Actual paid on confirmation (Figure A4 step 5).
10	Cash & Bank → Finance	Sync	Payment voucher posting (Figure A4 step 6); interest, FD and contra vouchers.
11	Budget → Finance	Event	Fund receipt voucher; transfer voucher when Internal pays for a Government item; surrender.
12	HR → Finance	Event	Salary accrual journal at VC approval: expense, employer share, every deduction payable, net salary payable.

DEVELOPER NOTE

- **Rule 1.** A synchronous call goes through the receiving module’s public interface and runs inside the caller’s transaction. The receiver may throw; the caller’s transaction rolls back.
- **Rule 2.** An event is a small immutable record (ids and amounts, never entities). It is written to the event registry in the publisher’s transaction and delivered after commit. A handler that fails is retried; a handler that succeeds marks the event complete. Every handler checks the event id before acting, so a redelivery is harmless.
- **Rule 3.** No module reads another module’s tables, in Java or in SQL. Reports read views in the reporting schema.
- **Rule 4.** A synchronous call never crosses from the finance group into HR, Examination or Inventory. Those modules call Finance-group modules; the reverse direction is always an event. This keeps the dependency arrows pointing one way (Figure A3).

C4 The shared kernel

The ten things every module needs and none may build for itself. Each closes a seam ACC-005 found between the BRDs.

Component	What it gives every module	Closes
Financial year	One master (April–March), one open year for posting, close statuses per module kept separate from the platform status	`X-15`
Masters	Fund (GOV, INT, RES if confirmed), Budget Head, Expense Head with its mapped head, bank and cash accounts, tax rate master with effective dates and Government Order reference	`X-17` `Q-HR-03`
Workflow and approval matrix	Generic maker → verifier → approver with configurable chains (Payroll → FO → VC → FO → Cash & Bank), amount bands that are half-open and most-specific-wins, delegation with audit, return and reject with reasons, and segregation of duties on the actor list	`GAP-12` `GAP-22` `Q-X-03` `X-03`
Audit trail	Envers history plus the audit_log row written by the workflow engine in the same transaction; a per-document timeline screen	Every BRD’s audit section
Documents	Upload, hash, link to a record, pre-signed download, retention; one implementation over MinIO	`Q-X-04`

Component	What it gives every module	Closes
Numbering	Gap-free number per series per financial year (PV/2026-27/000412), allocated under lock at posting, never at draft	`Q-FIN-05` `FIN §25`
Payee master	One record for vendors, agencies, examiners (Vendor Type 3) and refund recipients with encrypted bank details and maker-checker on changes; employees stay in HR with the same encryption	`X-08` `Q-EXM-06` `Q-HR-14`
Posting-rule engine	Event type → debit/credit templates with named amounts, versioned, Finance-approved; refuses a rule that does not balance (API-004 GL §2)	`Q-CB-01` `X-06`
Notifications	Outbox, templates, e-mail now, SMS or portal inbox later	—
Period control	Monthly soft lock, yearly hard lock, configured prior-period adjustment path	`Q-FIN-08` `BUD-BR-013`

C5 Data architecture

IN SIMPLE WORDS

One database, so that one transaction and one backup cover everything. One schema per module, so that ownership is visible and enforceable. Three append-only ledgers, so that every balance can be re-derived from its movements. Foreign keys only into the core schema, so that modules stay separable.

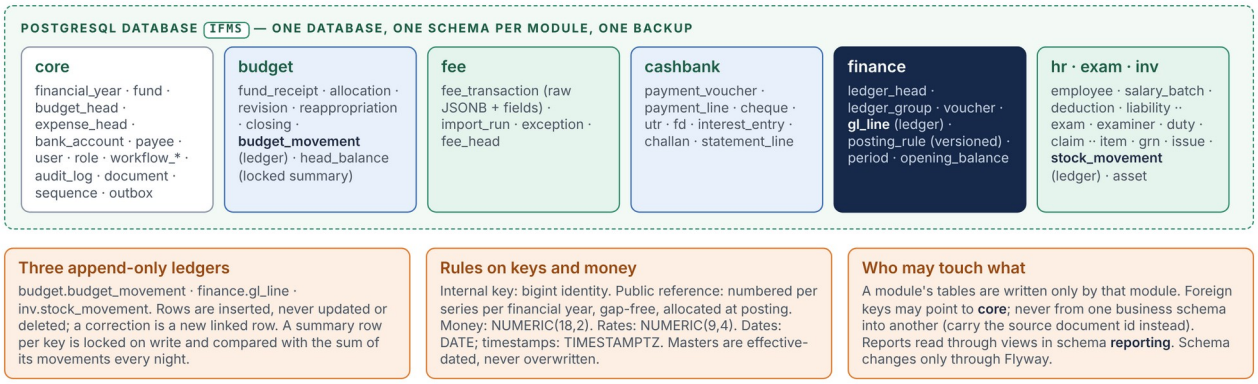


Figure C5 — Data layout. Table names are indicative; the LLD defines them. Bold names are the three ledgers.

DEVELOPER NOTE

- **Keys.** Internal: bigint identity, never exposed. Public: the gap-free reference number. Cross-module references carry the source document's public reference and id, not a foreign key.
- **Types.** Money NUMERIC(18,2); rates NUMERIC(9,4); quantities NUMERIC(14,3); dates DATE; moments TIMESTAMPTZ; enumerations as short text with a check constraint, not integers.
- **Effective dating.** Masters that change over time (pay matrix cells, tax rates, DA percentage, approval bands) are rows with from/to dates; the salary run reads the state on the salary month (`HR-BR-004`).
- **Summary rows.** One per key (financial year × fund × head; store × item; ledger head × period). Locked when written, compared with the sum of movements every night, difference reported before the next working day.
- **Volume.** Roughly two hundred thousand GL lines a year at this size; no partitioning needed for the first five years; indexes on (financial year, fund, head) and (ledger head, voucher date) carry every report.
- **Grants.** The API connects with one role per schema for writes and a read-only role for reports, so the "no cross-module reads" rule is enforced by the database as well as by the build.

C6 Security architecture

IN SIMPLE WORDS

The browser holds only cookies it cannot read. The web app turns a cookie into a bearer token. The API trusts only the bearer token, checks the role on every endpoint, and then checks the thing roles cannot express: that this person has not already acted on this document.

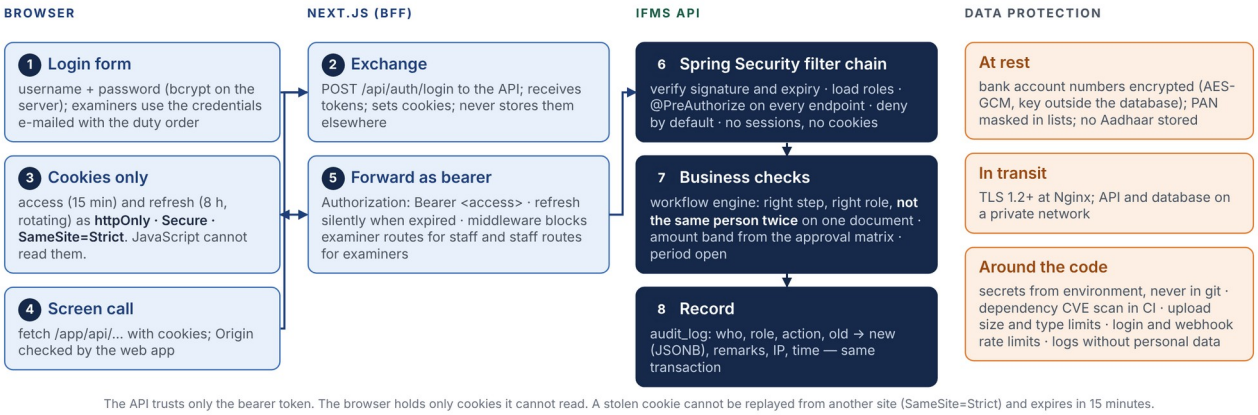


Figure C6 — Login, request and data-protection controls. The examiner portal follows the same flow with its own role and its own route group.

Role	Portal	May
BUDGET_MAKER, BUDGET_CHECKER	ERP	Create and verify fund receipts, allocations, revisions, re-appropriation, closing proposals
FINANCE_OFFICER	ERP	Verify and forward salary batches, bills, claim lists; approve closings per matrix; approve exceptions below the VC band
VICE_CHANCELLOR	ERP	Final approval per matrix; deviation decisions above a head
REGISTRAR	ERP	Approval matrix; examiner capping override
CASHBANK_USER	ERP	Payment vouchers, confirmation with UTR/cheque, bank accounts, FDs, interest, challans, statement upload
PAYROLL_USER	ERP	Employees, service book, salary batches, loans
ACCOUNTS_USER, FINANCE_VERIFIER, FINANCE_APPROVER	ERP	Chart of accounts, manual journals with maker-checker, opening balances, period close, posting rules
EXAM_USER (COE)	ERP	Academic masters, exams, examiners, duties, claim verification, cumulative list
STORE_USER, DEPARTMENT_USER	ERP	Requisitions, GRN, issues, returns, physical verification
RECEIVABLES_USER	ERP	Import runs, exception queue, manual allocation
AUDITOR	ERP	Read everything, change nothing, drill down from any figure
ADMIN	ERP	Users, roles, masters, configuration; every action audited; cannot approve a document
EXAMINER	Examiner	Own duties, acceptances, claims, bills and payment status only

WATCH OUT

- Roles are necessary but not sufficient. A user with both FINANCE_OFFICER and VICE_CHANCELLOR roles still cannot verify and approve the same batch (`GAP-22`); the workflow engine checks the actor list.
- Changes to an employee's or payee's bank account are maker-checker with a cooling period before the next

payment uses them (`Q-HR-14`).

- The admin role can create users and assign roles but cannot act in any workflow; an admin who needs to approve something is a separate user with that role.

C7 Non-functional targets

Proposed targets. They are deliberately modest for a system of this size and are confirmed with the hosting decision.

Area	Target	How the design meets it
Responsiveness	Any screen under 2 s at the 95th percentile with 50 concurrent users	Server-rendered first paint; paginated registers; indexes per report; no N+1
Payroll	Salary batch for 500 employees computed in under 3 minutes; slips generated in the background	Batch computed in one job; slips from the outbox; progress visible
Statements	Trial balance, I&E and Balance Sheet in under 5 s for a year	Set-based SQL over the ledger with period summaries
Fee import	5,000 records processed in under 2 minutes	Bulk insert with on-conflict de-duplication; exceptions queued, not blocking
Availability	99 % during working hours; maintenance evenings and weekends	Health checks, container restarts, one standby plan
Recovery	RPO 15 min, RTO 4 h	WAL archive, nightly dump, off-site copy, rehearsed restore
Retention	10 years for documents and ledgers; audit trail never purged	Append-only ledgers; MinIO retention policy; archival export yearly
Integrity	Dr = Cr per voucher per fund, always; summary = sum of movements, verified nightly	Database constraints; tie-out job with report
Concurrency	Two approvers on the same head never overdraw it	Row lock on the summary; commitment at approval
Auditability	Every state change has who, role, when, old, new; every figure drills to a document	Envers + audit_log; views keep source references
Locale	Indian number grouping, ₹, Asia/Kolkata, financial year April–March, English UI with Hindi labels on printed documents where required	One formatting module; report templates per language
Browsers	Current Chrome and Edge; mobile browsers for the examiner portal	Responsive layouts for the examiner routes only

C8 Environments and delivery pipeline

IN SIMPLE WORDS

A change is a pull request. The pipeline builds it, runs every test level from B8, builds a container image and deploys it to UAT. After the BA signs off a phase, the same image is tagged for production. Database changes travel inside the image as Flyway files and run at start-up.

#	Step	Detail
1	Pull request on a short-lived branch	Trunk-based; small changes; review by a second developer and the ECC review agents; CLAUDE.md rules apply

2	Continuous integration	Compile; unit, data-access and module tests (Testcontainers); boundary verification; OpenAPI snapshot diff; frontend type-check, lint and build; dependency vulnerability scan; year replay
3	Image build	Two images (api, web), tagged with the commit; signed; pushed to the registry
4	Deploy to UAT	Compose pull and restart; Flyway migrates on start, Hibernate validates; smoke test calls health and one endpoint per module
5	Phase demo and sign-off	BA and client on UAT; defects go back to step 1
6	Promote to production	Same image tagged release; maintenance window; backup taken first; migrations forward-only; rollback = previous image (schema changes are expand/contract so the previous image still runs)
7	After release	Health and error alerts for 24 hours; release note to the BA with the BRD rules it implements

Part D — Decisions register

Sixteen decisions in the format of an architecture decision record: the decision, why, what else was considered, what it costs, and its status. Marking one Approved is the reviewer's act; nothing is built on a Proposed decision beyond phase 0.

D-01 One deployable backend with seven modules inside (modular monolith)	Proposed
Decision	The API is one Spring Boot application with hard module boundaries (D-04). The four service names in API-004 (finance-api, hr-api, academic-api, ops-api) become the four module groups inside it.
Why	A payment confirmation must update the voucher, the budget head and the ledger together or not at all. Inside one process that is one database transaction. Across services it is a saga with compensating actions, the hardest thing in distributed systems, bought for a university with tens of concurrent users. One deployable also means one pipeline, one log, one place to debug, and a team of one or two can run it.
Alternatives	Four services as named in API-004: cross-service transactions, four deployments, duplicated masters. Kept as module boundaries only. One service per BRD: the same problems, more of them. One flat codebase with no enforced boundaries: quick for a month, a tangle by phase 4. Rejected because enforcement (D-04) costs nothing.
Consequences	Atomic postings, simple operations, modules can still be split later along the same lines (the finance group must stay together). One release train for all modules; mitigated by feature flags per module. The process is sized for payroll runs and report generation; mitigated by background jobs and a second instance.
Needs	F-02

D-02 Java 21 and Spring Boot 3.4 for the backend	Proposed
Decision	The API is written in Java 21 (LTS) on Spring Boot 3.4 with virtual threads enabled.
Why	Exact decimal arithmetic in the language; declarative transactions and locking that map onto PostgreSQL; field-level audit history with one annotation; the common stack of government and PSU IT vendors, which matters for hand-over (F-06); a strict compile-and-test gate that makes AI-assisted development safe to rely on.
Alternatives	NestJS (TypeScript): fastest start for a React team; rejected on the number type, hand-built audit and vendor familiarity. Go: superb for throughput; verbose for eighty tables of CRUD, workflow and reports; no mature ORM or audit library. Python (Django or FastAPI): exact Decimal and a free admin; weaker typing and concurrency for payroll and reports; less typical for the client's vendors. .NET: equivalent; team skills and Linux-first hosting favour Java.
Consequences	Longer start-up and more memory than a Node process; irrelevant at this scale. Java's traps (lazy loading, N+1, open-session-in-view) are rules in CLAUDE.md and the ECC Java pack, and the build checks the detectable ones.
Needs	—

D-03 PostgreSQL 16 as the only database	Proposed
Decision	One PostgreSQL 16 database holds every table; no second data store for business data.
Why	NUMERIC money; deferrable constraint triggers that enforce Dr = Cr per voucher per fund at commit; row locks that do not block readers; JSONB for raw payloads and audit snapshots; recursive queries for the chart of accounts; partial unique indexes for business rules; free and portable; routine backups and point-in-time recovery.
Alternatives	MySQL / MariaDB: no deferrable constraint triggers, weaker JSON. Oracle / SQL Server: licence and container friction; reconsidered only if the host already provides one (F-01). MongoDB: no referential integrity; reporting over documents is painful.
Consequences	The team must know PostgreSQL specifics (locks, vacuum, index types); covered by the rule packs and the LLD. Everything that is not business data (files) lives in MinIO (D-11).
Needs	F-01 may add a constraint

D-04 Spring Modulith for module boundaries and in-process events	Proposed
Decision	Each module is a top-level package with a declared public API. Spring Modulith verifies in the build that no module reaches into another's internals. Module-to-module notifications use its event publication registry, which stores events in the database in the publisher's transaction and re-delivers them after a crash.
Why	A boundary that is tested cannot erode. The registry is a ready-made outbox, so events are never lost and never need a broker. Module tests boot one module at a time and run in seconds.
Alternatives	Plain packages with ArchUnit rules: works for boundaries; events would need a hand-written outbox. A message broker (Kafka, RabbitMQ): infrastructure nobody needs at this scale (D-12).
Consequences	Modulith is younger than Spring itself (1.x since 2023); it is a thin layer over standard Spring, and the fallback (ArchUnit plus an own outbox) keeps the same code structure. Event handlers must be idempotent; the rule is in C3.
Needs	—

D-05 Next.js for both portals, TypeScript client generated from OpenAPI	Proposed
Decision	One Next.js (App Router) application serves the ERP portal and the Examiner portal as two route groups. Its API client and types are generated from the OpenAPI document that springdoc produces from the controllers.
Why	Two portals in one codebase with separate layouts and middleware; a server layer that keeps tokens out of the browser (D-06); types that cannot drift from the Java DTOs, with money as strings on both sides; a data-grid stack that fits register-heavy screens; fast first paint on slow links.
Alternatives	React with Vite: static hosting, but tokens in the browser and no server layer. Thymeleaf pages inside Spring: one deployable, but the grids these BRDs need do not fit. Angular: equally capable; the team's React experience decided.
Consequences	One more container (Node) to run. The discipline that the web app holds no business rule and no database access must be kept; it is a CLAUDE.md rule and a review item.
Needs	—

D-06 Session as httpOnly cookies set by the web app; the API accepts bearer tokens only

Proposed

Decision	The API issues a short-lived access token and a rotating refresh token. The web app stores both as httpOnly, Secure, SameSite=Strict cookies and forwards the access token as a bearer header. The API keeps no session.
Why	A token in browser storage is readable by any injected script; a cookie with these flags is not. The API stays stateless, scales by adding instances and is testable with a plain bearer token.
Alternatives	Server-side sessions (Spring Session): stateful API, sticky sessions or a session store. Tokens in browser storage: rejected for the reason above. University single sign-on: not available yet; the login interface leaves room for it.
Consequences	Refresh rotation and logout need care in the LLD; the web app checks the Origin header on every state-changing call.
Needs	—

D-07 One database, one schema per module, foreign keys only into core

Proposed

Decision	Schemas core, budget, fee, cashbank, finance, hr, exam, inv and reporting in one database. A business schema may reference core; it may not reference another business schema; it carries the source document's id and reference instead.
Why	One backup, one transaction, one connection pool; ownership is visible in every table name; database grants can enforce the "no cross-module reads" rule; modules remain separable later.
Alternatives	One flat schema: ownership invisible, rules unenforceable. A database per module: kills the atomic posting that D-01 exists for.
Consequences	Reports join across schemas only through views owned by reporting. No cascading deletes across modules, which is correct for a financial system.
Needs	—

D-08 Three append-only ledgers with locked summary rows and a nightly tie-out

Proposed

Decision	Budget movements, GL lines and stock movements are insert-only tables protected by database triggers. A summary row per key caches the balance; it is locked when written and compared with the sum of its movements every night.
Why	The BRDs require it (`BUD-BR-008` `FIN-BR-012` `CB-BR-008` `INV-BR-007`); auditors need every movement; a counter that is updated in place is a counter that can be wrong without anyone knowing.
Alternatives	Balance columns updated in place: fast and unauditable. Deriving every balance from movements on every read: correct but slow for availability checks; the summary row is the compromise, and the tie-out keeps it honest.
Consequences	Corrections are reversal rows, which users must be trained on. Storage grows with activity; trivial at this volume.
Needs	—

D-09 Flyway migrations, forward-only; Hibernate validates and never updates

Proposed

Decision	The schema exists only as versioned SQL files applied by Flyway at start-up. Hibernate runs in validate mode in every environment.

Why	Version-controlled, reviewable SQL with checksums that stop silent edits; PostgreSQL features (deferrable triggers, partial indexes) are plain SQL; the application refuses to start if entities and tables disagree.
Alternatives	Liquibase: equivalent; the YAML and diff tooling are not needed. Hibernate automatic DDL: forbidden for a financial system.
Consequences	Renames follow expand/contract so the previous image still runs during a rollback. A migration is immutable once it has run on UAT.
Needs	—

D-10 Audit: Envers for entity history, audit_log for the business trail, same transaction

Proposed

Decision	Every master and workflow entity is audited by Hibernate Envers. The workflow engine writes one audit_log row per action (who, role, action, document, old → new, remarks, IP, time) in the same transaction as the change.
Why	Each BRD asks for old value, new value, user, role and timestamp on everything. Envers gives field-level revisions for free; audit_log gives the business meaning and the document timeline screen.
Alternatives	Database triggers only: no user and role context without session variables; harder to read. Application logs only: not queryable, not durable, not evidence.
Consequences	Storage; one timeline screen per document; audit tables excluded from the nightly tie-out.
Needs	—

D-11 MinIO (S3 API) for attachments and generated documents

Proposed

Decision	Files live in a versioned MinIO bucket with a retention policy; PostgreSQL keeps the key, size, SHA-256, uploader and the link to the record. The browser uploads and downloads through pre-signed links.
Why	Every BRD attaches documents but the DMS module was dropped (`Q-X-04`); the S3 API means the same code works against any provider's object storage if hosting changes; versioning and retention are built in.
Alternatives	Files on the API container's disk: lost on replacement, no versioning. BYTEA columns in PostgreSQL: bloats backups and the buffer cache. Provider S3: the natural choice if F-01 picks cloud; a configuration change.
Consequences	MinIO is licensed AGPL-3.0; used unmodified as a separate service this is acceptable in most organisations, but it is for the reviewer to confirm (F-07). Permissively licensed alternatives: SeaweedFS (Apache-2.0) or the provider's S3.
Needs	F-07

D-12 No Redis and no message broker in phase 1

Proposed

Decision	Idempotency keys, rate limits, job locks, the outbox and the event registry live in PostgreSQL. No cache service, no broker.
Why	At this volume PostgreSQL does all of it; every extra service is one more thing to back up, patch, monitor and explain at hand-over.
Alternatives	Redis plus a queue from day one: a habit from the Node world, not a need here.
Consequences	Triggers that would bring Redis in later: a second API instance with heavy caching, more than fifty thousand webhook calls a day, or sub-second dashboards. None is expected in the first two years.

Needs	F-05

D-13 JPA for writes, hand-written SQL for reports; jOOQ held in reserve

Proposed

Decision	Commands (create, verify, approve, post, pay) use JPA entities with locks. Statements and registers are SQL files executed through Spring JdbcClient, read-only.
Why	Commands are object graphs with invariants; statements are set-based arithmetic over the ledger. Plain SQL for the latter is clearer, faster and can be reviewed by the BA and the accountant. jOOQ is adopted only if the report SQL outgrows plain files.
Alternatives	JPQL for reports: awkward grouping, poor control of the plan. jOOQ everywhere: good, but a code-generation step for little gain at the start.
Consequences	Two ways to touch the database, with one rule: SQL reads, never writes.
Needs	—

D-14 Documents generated on the server with JasperReports (PDF) and Apache POI (Excel)

Proposed

Decision	Printed vouchers, cheques, challans, duty orders, salary slips, Utilisation Certificates and registers are rendered by the API from templates.
Why	These need exact layouts, bilingual labels and signature blocks; both libraries are mature, free, and run in the same JVM; templates are files the BA can review against the samples (`Q-X-05`).
Alternatives	Browser printing: unreliable layouts. Headless Chrome in a container: works; kept as the fallback for HTML-heavy reports.
Consequences	Template work per document type in each phase; samples must be collected first.
Needs	—

D-15 Scheduled jobs inside the API process, guarded by ShedLock

Proposed

Decision	The dozen small jobs (nightly tie-out, interest accrual, outbox delivery, utilisation alerts, FD maturity reminders, retention clean-up, import polling) run inside the API on Spring's scheduler with a database lock per job.
Why	Keeps one deployable (D-01) while still allowing two instances; nothing to install.
Alternatives	A separate worker container: the same code as a second process; only if payroll runs ever starve the API. Quartz: heavier; persistent triggers not needed.
Consequences	Long jobs must be chunked so a restart loses little; job runs are logged to a table visible on an admin screen.
Needs	—

D-16 Posting rules are versioned data, not code (carried from API-004)

Proposed

Decision	An event type maps to debit and credit line templates with named amounts, stored with a version and an effective date, approved by Finance through maker-checker. The engine refuses a rule whose lines cannot balance.

Why	The open BA questions (`Q-FIN-02` `Q-FIN-03` `Q-CB-01`) will change postings after go-live; a rule change must not need a release. The ACC-005 Part D year becomes the acceptance test for every rule set.
Alternatives	Hard-coded postings per module: every BA answer becomes a code change and a deployment.
Consequences	A rule editor screen with maker-checker; a rule version is stamped on every voucher it produced.
Needs	—

Part E — Build order

Seven phases in dependency order, so that no phase has to mock a table that does not exist yet. This is a sequence, not a calendar; dates follow once the team and the hosting (F-01, F-06) are known.

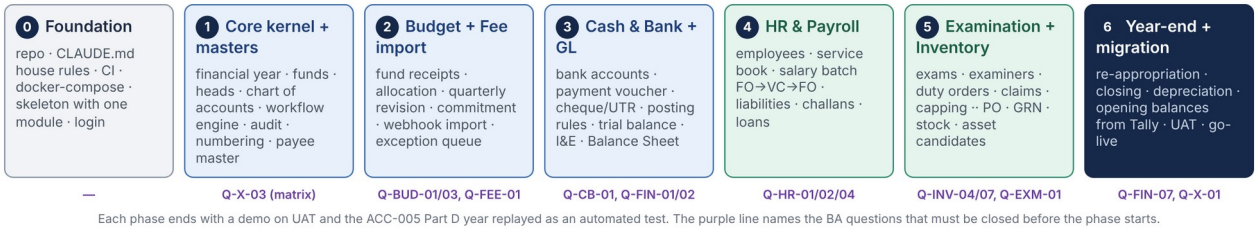


Figure E1 — Build order. Each phase ends with a demo on UAT and the ACC-005 Part D year replayed as an automated test over everything built so far.

Phase	Delivers	Must be closed first	Done when
0	Repository, CLAUDE.md house rules, CI pipeline, docker-compose, application skeleton with the core module and login, the two portals' shells	D-01 to D-07 approved	A developer can clone, run, log in and see an empty dashboard; CI is green on an empty change
1	Core kernel and masters: financial year, funds, budget heads, expense heads, chart of accounts, workflow engine and approval matrix, audit, numbering, documents, payee master, period control	`Q-X-03` (matrix sheet), `Q-FIN-01` (chart of accounts format)	A fictional document can be created, verified and approved through a configured chain with the audit timeline and segregation of duties working
2	Budget and Fee Revenue Import: fund receipts, head-level allocation, quarterly revision, commitment, availability; webhook import, de-duplication, exception queue, classification	`Q-BUD-01` `Q-BUD-03` `Q-FEE-01`	Part D T03–T05, T07–T08 reproduce; an April import run shows the exception queue explaining every rupee of difference
3	Cash & Bank and the General Ledger: bank account hierarchy, payment voucher, cheque/UTR, interest and FDs, challans; posting rules, voucher posting, ledger, trial balance, I&E, Balance Sheet	`Q-CB-01` `Q-FIN-02` `Q-FIN-03`	The full Part D year posts and the trial balance, I&E and Balance Sheet agree to the rupee; Figure A4 runs end to end
4	HR & Payroll: employees, service book, pay matrix, salary batch through FO → VC → FO → Cash & Bank, deductions, liability register, challans, loans	`Q-HR-01` `Q-HR-02` `Q-HR-04`	One month's batch for the sample employees matches the register the Payroll section prepares today
5	Examination and Inventory: academic masters, exams and centres, examiner onboarding and portal, duty orders, claims, capping, cumulative list; items, PO, GRN, stock register, issues, asset candidates, depreciation	`Q-INV-04` `Q-INV-07` `Q-EXM-01` `Q-EXM-02`	An examiner accepts a duty from the e-mailed link and is paid through Cash & Bank; a GRN becomes stock, an issue becomes an expense, a computer becomes a depreciating asset

Phase	Delivers	Must be closed first	Done when
6	Year-end and migration: re-appropriation, surrender and carry-forward, period close, opening balance import from Tally, UAT hardening, cut-over rehearsal, go-live	`Q-FIN-07` `Q-X-01` `Q-BUD-13`	The Tally trial balance imports and balances; a rehearsal cut-over on UAT completes within the window; the client signs off

DEVELOPER NOTE

- Phases 2 and 3 can overlap once the kernel is stable, because Budget and the GL share only the kernel. Phases 4 and 5 depend on 3.
- Every phase adds its vouchers to the year-replay fixture; the fixture is the living acceptance test and the BA's best evidence.
- The eighteen critical BA questions from ACC-005 Part F are spread across the "must be closed first" column; none blocks phase 0 or 1.

Part F — Needs sign-off

Ten points only the reviewer, the client or PCS management can settle. Each has a proposed default so that phase 0 can start; a decision that changes a default also changes the decision it refers to.

F-01 Where does production run? — Open

Question	University server room, the UP State Data Centre, or a cloud provider? This decides VM sizing, backup targets, whether an existing database licence or monitoring stack must be used, and who patches the operating system.
Proposed default	Two VMs as in Figure C1 wherever the university's IT policy allows; cloud only if the client asks for it. PostgreSQL and MinIO as designed unless the host provides an equivalent.
Affects	D-03, D-11, C1, C7

F-02 One deployable, or the four services named in API-004? — Open

Question	API-004 names finance-api, hr-api, academic-api and ops-api. This document keeps them as module groups inside one deployable (D-01). Confirm, or state the reason four processes are required.
Proposed default	One deployable. The finance group stays together in either case.
Affects	D-01, D-04, C1, C3

F-03 Bank channels in phase 1 — Open

Question	File-based NEFT/RTGS and cheque printing only, with the HDFC payment API in phase 2 (`Q-CB-03`)? The bank file formats and the cheque layout are needed before phase 3.
Proposed default	Files and cheques in phase 1; one adapter per bank behind one interface; API later.
Affects	C2, Phase 3

F-04 Digital signature / e-Sign — Open

Question	Which documents must carry a digital signature at go-live (duty orders, cheques, vouchers, UCs) and through which provider (`Q-X-02`)?
Proposed default	Printed and signed in phase 1; e-Sign in phase 2 once the provider is chosen.
Affects	B7, C2

F-05 No cache or queue service in phase 1 — Open

Question	Accept that PostgreSQL carries locks, idempotency keys, the outbox and job scheduling (D-12), with Redis added only on the listed triggers?
Proposed default	Yes.
Affects	D-12, D-15

F-06 Who maintains the system after hand-over? — Open

Question	PCS under an annual contract, the university's IT cell, or a third party? The answer sets the training plan and confirms that Java and Spring skills will be available.

Proposed default	PCS maintains for the first year with a documented hand-over; the stack choice (D-02) already favours the widest skill pool.
Affects	D-02, B8

F-07 Licences — Open

Question	Every component chosen is open source. Most are Apache-2.0 or MIT; PostgreSQL has its own permissive licence; MinIO is AGPL-3.0. Confirm AGPL is acceptable for a service used unmodified, or choose SeaweedFS (Apache-2.0) or the host's S3.
Proposed default	MinIO, unmodified, as a separate container.
Affects	D-11

F-08 Monitoring and alerting stack — Open

Question	Actuator health and structured logs only in phase 1, with Prometheus and Grafana in phase 2, or whatever the host already runs?
Proposed default	Actuator and logs in phase 1; a dashboard stack when the host is known.
Affects	B7, C8

F-09 Cut-over approach — Open

Question	Opening balances on 1 April 2026 with monthly summaries for the months before go-live, or the go-live date itself (`Q-X-01` `Q-FIN-07`)? Phase 6 cannot be planned without it.
Proposed default	Opening trial balance on 1 April 2026 from Tally, monthly summaries until go-live, first IFMS statements for FY 2026-27.
Affects	Phase 6, C2

F-10 The next document — Open

Question	Proceed to the Low-Level Design (IFMS-DG-LLD-007: schemas, endpoints, screens, posting rules) once Parts B and D are approved, with the API-004 posting-rule sections moved into it?
Proposed default	Yes; the LLD is written phase by phase in the order of Part E, starting with the kernel.
Affects	Everything after this

HOW TO CLOSE THIS DOCUMENT

To approve: mark each decision in Part D Approved or write the change against its ID, answer F-01 to F-10 (the proposed default is enough), and sign the cover. Answers that change a decision are made in this document and it moves to v1.0; answers that change a BRD become change requests against that BRD, as in ACC-005.

Appendix — Technical glossary and document family

G1 Glossary

Term	Meaning here
Append-only ledger	A table whose rows are only ever inserted; a mistake is corrected by a new row that reverses the old one.
BFF (backend for frontend)	The web app's server side, which holds the session cookies and forwards calls to the API. It has no business logic.
Bearer token (JWT)	A signed, self-contained proof of identity sent with each API call; the API verifies the signature instead of looking up a session.
Deferrable constraint	A database rule checked at commit rather than after every row, so that a voucher's lines can be written one by one and still be required to balance.
Event registry / outbox	A table where a module writes "this happened" in its own transaction; a delivery job hands the record to the receiving module afterwards and retries until it succeeds.
Expand / contract	Renaming or restructuring in two steps (add the new, migrate, remove the old) so that the previous version of the application still runs during the change.
Flyway	The tool that applies numbered SQL migration files in order and records which have run.
Hibernate Envers	The library that keeps a history table for every audited entity with the user and time of each change.
httpOnly cookie	A cookie the browser sends with requests but does not let page scripts read.
Idempotency key	A unique id sent with a request so that the same request sent twice has the same effect as once.
JSONB	PostgreSQL's binary JSON column type; indexable; used for raw payloads and audit snapshots.
Modular monolith	One deployable application whose internal modules have enforced boundaries, as separate services would, without the network between them.
OpenAPI	The machine-readable description of the API from which the frontend client and types are generated.
Pre-signed link	A time-limited URL that lets the browser upload or download one file directly from the file store without passing through the API.
Row lock (SELECT … FOR UPDATE)	Locking one row so that a second transaction wanting the same row waits; used on budget, stock and ledger summary rows.
Segregation of duties	The rule that one person may act at only one step of one document, whatever roles they hold.
ShedLock	A small library that uses a database row as a lock so that a scheduled job runs on one instance at a time.
Spring Modulith	A Spring library that verifies module boundaries in the build and provides the in-database event registry.
Testcontainers	A test library that starts a real PostgreSQL in a container for the duration of the tests.
WAL archive, PITR	PostgreSQL's write-ahead log copied continuously so the database can be restored to any point in time.

G2 Document family

ID	Title	Status
IFMS-DG-ACC-002 v1.0	Accounting Developer Guide — Budget v1 and Salary v2.0 (printed)	Background; Part B superseded

ID	Title	Status
IFMS-DG-API-004	API and posting-rule design (service names, GL posting rules)	Posting design carried forward; service split revisited here (F-02)
IFMS-DG-ACC-005 v1.0	Accounting Developer Guide — All Modules: money flow, books and BA questions	Current; its Part F questions gate the phases in Part E
IFMS-DG-HLD-006 v0.1	High-Level Design — Services, Stack and How They Connect (this document)	Draft for approval
IFMS-DG-LLD-007	Low-Level Design — schemas, endpoints, screens, posting rules, per phase	Starts after approval (F-10)

— End of document IFMS-DG-HLD-006 v0.1 —